```
  def m2      # this method is protected
  end
  private
  def m3      # this method is private
  end
end
```

The `private` directive is the strictest; private methods can be called only from within the same instance. Protected methods can be called both in the same instance and by other instances of the same class and its subclasses.

Classes aren't the only organizing structure in Ruby. The other organizing structure is a *module*.

Modules

Modules are similar to classes in that they hold a collection of methods, constants, and other module and class definitions. Unlike with classes, you can't create objects based on modules.

Modules serve two purposes. First, they act as a namespace, letting you define methods whose names won't clash with those defined elsewhere. Second, they allow you to share functionality among classes. If a class *mixes in* a module, that module's methods become available as if they'd been defined in the class. Multiple classes can mix in the same module, sharing the module's functionality without using inheritance. You can also mix multiple modules into a single class.

Helper methods are an example of where Rails uses modules. Rails automatically mixes these helper modules into the appropriate view templates. For example, if you wanted to write a helper method that's callable from views invoked by the store controller, you could define the following module in the `store_helper.rb` file in the `app/helpers` directory:

```
module StoreHelper
  def capitalize_words(string)
    string.split(' ').map {|word| word.capitalize}.join(' ')
  end
end
```